CS 791 - Assignment 1 Report

Balaji Karedla, Harith S, Sandeep Reddy Nallamilli

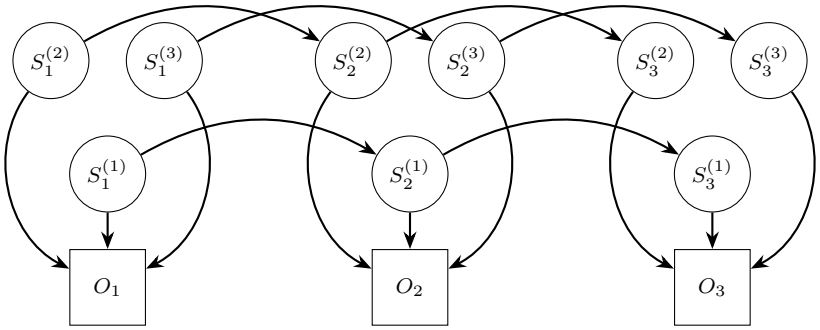
Contents

1 Factorial Hidden Markov Model (FHMM)

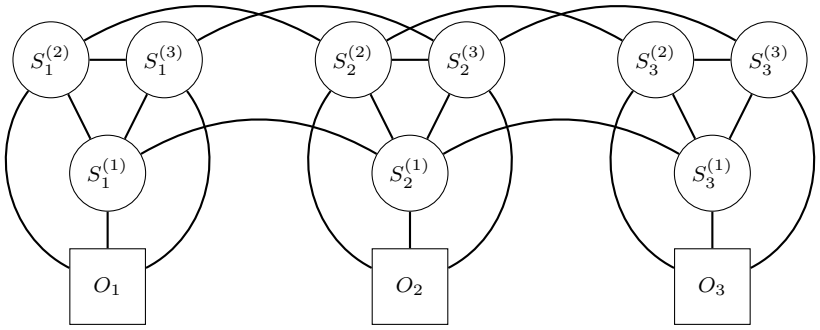
An FHMM is an extension of the standard Hidden Markov Model (HMM) that allows for multiple independent hidden state sequences to influence the observed data.

The problem of interest is to infer the marginals of the hidden states, and the most probable sequence of hidden states, given a sequence of observations in a Factorial Hidden Markov Model (FHMM).

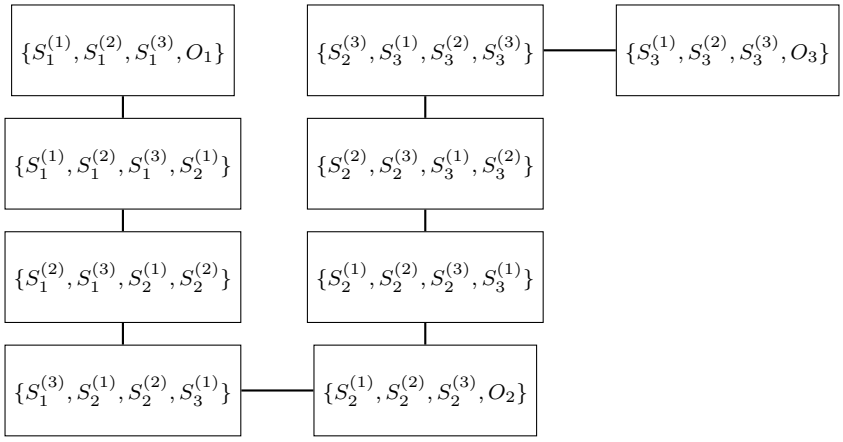
The problem involved making a junction tree out of the FHMM by triangulating it and then using the message passing algorithm to infer the marginals of the hidden states and the most probable sequence of hidden states given a sequence of observations.



The moralized graph looks like



The junction tree made out of the cliques after triangulation is a chain as shown below:



The message passing algorithm is now simple with just two passes, one from left to right and the other from right to left.

For including the prior of the observations, we merged the observations into the cliques by removing the observation nodes and considering the potentials over the cliques that include the observations.

In the junction tree, there are $M(T-1)+T$ cliques of size $M+1$ and all the cliques form a chain. Here, M is the number of factors and T is the number of time steps.

1.1 Z-Value Computation

For finding the z-value of the junction tree, the message passing algorithm runs over every clique for every assignment of the variable in the clique at max and hence the time complexity is $O(TMK^{M+1})$, assuming the translation of dict to the indices occurs in $O(1)$, where K is the number of states each hidden variable can take.

1.2 Marginals of the Hidden States

For the marginals of the hidden states, we ran the message passing algorithm forward and backward, and then computed the marginals for each hidden state by marginalizing over the cliques that contain the hidden state. This took $O(TMK^{M+1})$ time.

1.3 Most Probable Sequences of Hidden States

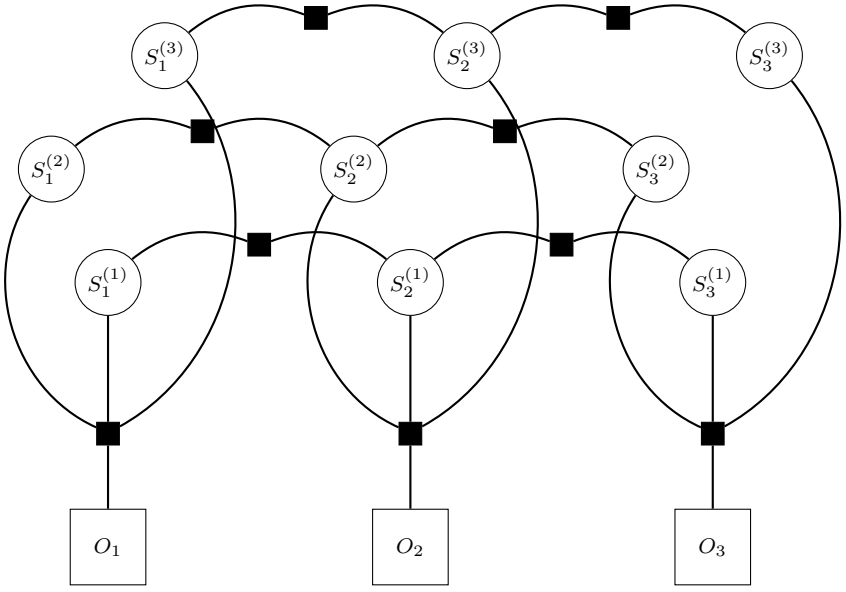
For the top-k most probable sequence of hidden states, we modified the message passing algorithm to each potential storing a k-heap of assignments and their probabilities and merging the k-heaps appropriately during message passing. This took $O(TMK^{M+1}k\log k)$ time.

2 Loopy Belief Propagation

Since exact inference is often computationally expensive, belief propagation (BP) is widely used as an approximate inference algorithm. The procedure starts by constructing a factor graph. We maintain beliefs for all variable nodes and factor nodes, and update them iteratively using a set of message-passing equations. On tree-structured graphs this yields exact marginals, while on graphs with loops, the same procedure (called loopy belief propagation) is only approximate.

The key idea is that if BP converges, it often converges to the correct marginal probabilities (exactly so in trees; approximately so in loopy graphs). However, because this is an iterative approximation algorithm, convergence is not guaranteed in general.

The order of message updates does not strictly matter, as long as all beliefs and messages are eventually updated. In my implementation, I update factor nodes first, then transition nodes, then send messages to variables, update variable beliefs, and finally send messages from variables back to factor and transition nodes. At each iteration, to avoid double counting, the contribution from the previous message that was already incorporated into a belief is divided out before incorporating new messages.



Let the number of states be `K`, number of states be `N`, number of factors be `M` and number of observations be `T`.

- The function `updateFactorNodeBeliefs` has a time complexity of $O(TMK^M)$
- The function `updateTransitionNodeBeliefs` has a time complexity of $O(TMK^2)$
- The function `messageToSendFromFactor` has a time complexity of $O(MK^M)$
- The function `messageToSendFromTransition` has a time complexity of $O(K^2)$
- The resulting complexity of function `sendMessagesToVariables` is $O(TM^2K^M + TMK^2)$

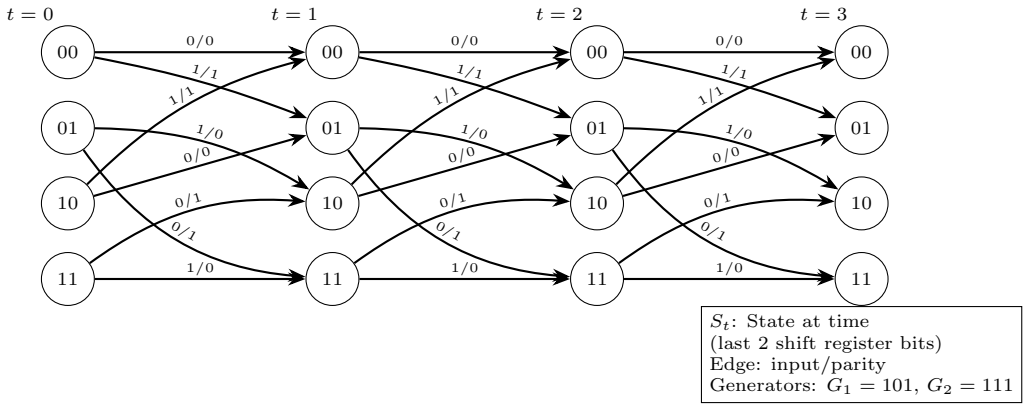
- The function `updateVariableBeliefs` has a time complexity of $O(TMK)$
- The function `sendMessagesToFactorsNodes` has a time complexity of $O(TMK)$
- The function `sendMessagesToTransNodes` has a time complexity of $O(TMK)$
- Checking convergence has an order of $O(TMK)$

Therefore, the overall time complexity of the algorithm is $O(MF^2K^F + MFK^2)$ assuming O(1) iterations for convergence.

3 Turbo Codes

3.1 Generating the Trellis

The sequence of output bits is generated by computing the parity of the product between the generator polynomial and the contents of the shift register. If the shift register has length k, the encoder requires 2^{k-1} distinct states, since each new bit is obtained by appending the input bit to the register and then applying the parity operation. For every state and input pair, we compute both the next state of the register and the corresponding output parity bit. Using this information, we construct the trellis diagram over the full sequence of input bits.
Below is an example trellis with a shift register of length 3, generators 5 and 7, and a sequence of 4 bits. (Start state for any sequence is 00)



3.2 Viterbi

The algorithm begins in state 0 with an initial cost of 0, while all other states are assigned an infinite cost. Here, the cost represents the negative log likelihood of the most probable sequence leading to a particular state.
During the forward pass, the algorithm iterates over each time step (corresponding to one bit of the original sequence) and evaluates all possible states at that step. For every possible input bit, it computes the likelihood of observing both the systematic and parity bits. These likelihoods are converted to costs and added to the cumulative cost of reaching the current state. If this new path yields a lower cost than any previously recorded path to the next state, the cost is updated along with a backpointer to record the optimal predecessor.
In the backtracking phase, the algorithm identifies the final state with the minimum cost (or the maximum likelihood). Starting from this state, it follows the backpointers in reverse through the trellis and reconstructs the most probable sequence of input bits.

The time complexity of the algorithm is $O(T * S)$ where T is the length of the original string or the number of timestamps and S is the total number of states (in this case 2^{k-1}) where k is the length of the shift register.

3.3 BCJL and Turbo Codes

In turbo decoding, a posteriori probabilities (APPs) are estimated with the BCJR algorithm, starting from uniform priors ($P(b=0) = P(b=1) = \frac{1}{2}$). For each received sequence, BCJR computes log-likelihood ratios (LLRs):

$$LLR_i = \log \frac{P(u_i = 1 \mid \mathbf{y})}{P(u_i = 0 \mid \mathbf{y})},$$

where u_i is the transmitted bit and $\mathbf{y}$ the channel output.
Each decoder then forms *extrinsic information* by removing the prior contribution:

$$LLR_i^{\text{extrinsic}} = LLR_i - LLR_i^{\text{prior}},$$

$$LLR_i^{\text{prior}} = \log \frac{P(u_i = 1)}{P(u_i = 0)}.$$

These extrinsic values are interleaved and passed to the other decoder as new priors. Iterating this exchange gradually refines the APPs.
Convergence is monitored via the maximum LLR change,

$$\Delta = \max_i \left| LLR_i^{(t)} - LLR_i^{(t-1)} \right|,$$

stopping early if Δ falls below a threshold, or otherwise after a fixed maximum number of iterations.

3.4 Results

We compared the decoding performance of three algorithms: the Viterbi decoder, the BCJR (MAP) decoder, and the Turbo decoder. The cumulative average BERs across all experiments are summarised below (and in results.txt):

- Viterbi decoder: **3.90%**
- BCJR decoder: **3.61%**
- Turbo decoder: **0.0067%**

In terms of BER, BCJL performs slightly better than Viterbi on average. This might be because BCJL gives the probability for each bit individually. The average length of a matched string is better in Viterbi, probably because Viterbi outputs the most probable string.
Since the turbo code always has almost zero errors, applying Viterbi on the final APP helps but does not improve the BER to any significant degree.

4 Contributions

- **Message Passing on Factorial HMMs:** Balaji Karedla
- —textbfLoopy Belief Propagation: Harith S
- **Turbo Codes:** Sandeep Reddy Nallamilli

5 LLM Usage

- **Message Passing on Factorial HMMs (Q1)**
  - Just a bit of CoPilot auto-completion for the idea I had in mind
  - Never used a prompt to generate code or copied code from LLMs explicitly
- **Loopy Belief Propagation (Q2)**
  - Initialising factor beliefs (partly, was debugged with LLMs)
  - `assignmentToIndex` function
  - `normalise` function
  - Divide by 0 handling
  - Checking for convergence
- **Turbo Codes (Q3)**
  - Clarified some doubts in how the BCJR algorithm worked
  - Debugging numerical overflow and underflow errors
  - Creating the tikz diagram for the report
  - Other latex formatting